

Ex. No: 11a

Experiment Title: FIFO PAGE REPLACEMENT

Aim:

To find out the number of page faults that occur using First-in First-out (FIFO) page replacement technique.

Procedure:

1. Declare the size with respect to page length.
2. Check the need of replacement from the page to memory .
3. Check the need of replacement from old page to new page in memory.
4. Form a queue to hold all pages.
5. Insert the page require memory into the queue.
6. Check for bad replacement and page fault.
7. Get the number of processes to be inserted.
8. Display the values.

Program:

Filename: `fifo.c`

```
#include <stdio.h>
#define MAX_PAGES 100
#define MAX_FRAMES 10

int main() {
    int pages[MAX_PAGES], frames[MAX_FRAMES];
    int n, frameSize, pageFaults = 0;
    int front = 0;
    int i, j, k, found;
```

```

printf("Enter the size of the reference string: ");
scanf("%d", &n);

printf("Enter the reference string (space-separated page
numbers):\n");
for (i = 0; i < n; i++) {
    scanf("%d", &pages[i]);
}

printf("Enter the page frame size: ");
scanf("%d", &frameSize);

for (i = 0; i < frameSize; i++) {
    frames[i] = -1;
}

printf("\n%-10s%-20s%-15s\n", "Page", "Frames", "Page Fault");
printf("-----\n");

for (i = 0; i < n; i++) {
    found = 0;
    for (j = 0; j < frameSize; j++) {
        if (frames[j] == pages[i]) {
            found = 1;
            break;
        }
    }

    printf("%-10d", pages[i]);

    if (!found) {
        frames[front] = pages[i];
        front = (front + 1) % frameSize;
        pageFaults++;
    }

    for (k = 0; k < frameSize; k++) {
        if (frames[k] != -1)

```

```

        printf("%d ", frames[k]);
    else
        printf("- ");
}

if (!found)
    printf("%15s\n", "Yes");
else
    printf("%15s\n", "No");
}

printf("\nTotal number of page faults: %d\n", pageFaults);
return 0;
}

```

□

Sample Input and Output:

□ Enter the size of the reference string: 12
Enter the reference string (space-separated page numbers):
1 3 0 3 5 6 3 0 1 2 1 2
Enter the page frame size: 3

Page	Frames	Page Fault
1	1 - -	Yes
3	1 3 -	Yes
0	1 3 0	Yes
3	1 3 0	No
5	5 3 0	Yes
6	5 6 0	Yes
3	5 6 3	Yes
0	0 6 3	Yes
1	0 1 3	Yes
2	0 1 2	Yes
1	0 1 2	No
2	0 1 2	No

Total number of page faults: 9

**Result:**

FIFO page replacement program has been successfully executed.

Ex. No: 11b

Experiment Title: LRU

Aim:

To write a c program to implement LRU page replacement algorithm.

Procedure:

- 1: Start the process.
- 2: Declare the size.
- 3: Get the number of pages to be inserted.
- 4: Get the value.
- 5: Declare counter and stack.
- 6: Select the least recently used page by counter value.
- 7: Stack them according to the selection.
- 8: Display the values.
- 9: Stop the process.

Program:

Filename: LRU.c

```
#include <stdio.h>
```

```
int main() {  
  
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3};  
  
    int n = 10;  
  
    int frames = 3;  
  
    int frame[10], recent[10];  
  
    int pageFaults = 0, time = 0;  
  
    for (int i = 0; i < frames; i++) {  
        frame[i] = -1;  
    }  
  
    printf("Page\tFrames\n");  
  
    for (int i = 0; i < n; i++) {  
        int found = 0;  
  
        for (int j = 0; j < frames; j++) {  
            if (frame[j] == pages[i]) {  
                found = 1;  
                recent[j] = time++;  
                break;  
            }  
        }  
  
        if (found == 0) {  
            int pos = 0, min = recent[0];  
            for (int j = 1; j < frames; j++) {  
                if (recent[j] < min) {
```

```

        min = recent[j];

        pos = j;
    }

}

frame[pos] = pages[i];

recent[pos] = time++;

pageFaults++;

}

printf("%d\t", pages[i]);

for (int j = 0; j < frames; j++) {

    if (frame[j] != -1)

        printf("%d ", frame[j]);

    else

        printf("- ");

}

printf("\n");

}

printf("\nTotal Page Faults = %d\n", pageFaults);

return 0;

}

```

□

Sample Input and Output:

□ `int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3};`

```
int frames = 3;
```

Page	Frames
------	--------

7	7 - -
---	-------

0	7 0 -
---	-------

1	7 0 1
---	-------

2	0 1 2
---	-------

0	0 1 2
---	-------

3	3 1 2
---	-------

0	3 0 2
---	-------

4	4 0 2
---	-------

2	4 0 2
---	-------

3	3 0 2
---	-------

Total Page Faults = 8



Result:

LRU page replacement program has been successfully executed.

Ex. No: 11c

Experiment Title: OPTIMAL

Aim:

To write a c program to implement an Optimal page replacement algorithm.

Procedure:

- 1: Start the process.
- 2: Declare the size.
- 3: Get the number of pages to be inserted.
- 4: Get the value.
- 5: Declare counter and stack.
- 6: Select the least recently used page by counter value.
- 7: Stack them according to the selection.
- 8: Display the values.
- 9: Stop the process.

Program:

Filename: Optimal.c

```
#include <stdio.h>

int main() {

    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3};

    int n = 10;

    int frames = 3;

    int frame[10];

    int pageFaults = 0;

    for (int i = 0; i < frames; i++) {
```



```
        frame[i] = -1;
    }

    printf("Page\tFrames\n");

    for (int i = 0; i < n; i++) {

        int found = 0;

        for (int j = 0; j < frames; j++) {

            if (frame[j] == pages[i]) {

                found = 1;

                break;

            }

        }

        if (found == 0) {

            pageFaults++;

            int empty = -1;

            for (int j = 0; j < frames; j++) {

                if (frame[j] == -1) {

                    empty = j;

                    break;

                }

            }

            if (empty != -1) {

                frame[empty] = pages[i];

            } else {
```

```

        int farthest = -1, pos = -1;
        for (int j = 0; j < frames; j++) {
            int k;
            for (k = i + 1; k < n; k++) {
                if (frame[j] == pages[k])
                    break;
            }
            if (k > farthest) {
                farthest = k;
                pos = j;
            }
        }
        frame[pos] = pages[i];
    }
}

printf("%d\t", pages[i]);
for (int j = 0; j < frames; j++) {
    if (frame[j] != -1)
        printf("%d ", frame[j]);
    else
        printf("- ");
}

printf("\n");

```

```

    }

    printf("\nTotal Page Faults = %d\n", pageFaults);

    return 0;
}

□

```

Sample Input and Output:

```

□int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3};

int frames = 3;

```

Page	Frames
------	--------

7	7 - -
---	-------

0	7 0 -
---	-------

1	7 0 1
---	-------

2	0 1 2
---	-------

0	0 1 2
---	-------

3	0 3 2
---	-------

0	0 3 2
---	-------

4	4 3 2
---	-------

2	4 3 2
---	-------

3	4 3 2
---	-------

Total Page Faults = 7

□

Result:

Optimal page replacement program has been successfully executed.